OOP

The Perfect Interview Guide

From First Principles to Interview Confidence

14 Sections · 50+ Interview Q&As · Python Code · Cheat Sheets · Revision Tables

Designed for complete beginners · Internship & Entry-Level interviews

1 OOP From First Principles

What Is Programming?

Programming is giving instructions to a computer to solve a problem. Early programs were just lists of instructions, one after another — like a recipe. This worked fine for small programs.

The Problem With Procedural Programming

As programs grew bigger, the old approach — called **procedural programming** — started causing serious problems:

- Data and functions were completely separate. A function could accidentally change any data.
- The same data had to be passed to dozens of functions — messy and error-prone.
- Adding a new feature often broke existing code.
- Code was nearly impossible to reuse in other programs.

Procedural Code — The Problem

```
# PROCEDURAL way — data and functions are separate (messy)
student_name = "Ali"
student_gpa  = 3.7
student_roll = "BSDSF24A025"

def print_student(name, gpa, roll):   # must pass everything every time
    print(f"{name} | {roll} | GPA: {gpa}")

def update_gpa(gpa, new_gpa):          # nothing stops invalid values
    return new_gpa

student_gpa = update_gpa(student_gpa, -5)   # GPA = -5 !! No protection
```

How OOP Solves This

OOP (Object-Oriented Programming) says: instead of having data floating around separately, **bundle the data AND the functions that work on it together into one unit** called a **class**.

- Data is protected inside the class — no accidental changes
- Functions (called methods) live next to the data they work on
- Code can be reused through inheritance
- Big programs become manageable through modular objects

The Real-World Analogy

Think of a **smartphone**. It bundles together:

- Data: your contacts, photos, settings (the attributes)
- Functions: call, text, take photo (the methods)
- Protection: you can't directly access the circuit board — you use the screen (encapsulation)
- Reuse: an iPhone and Android are both phones — same concept, different implementation (inheritance)

OOP models your code the same way. Every real-world thing can become an object.

Q Why was OOP created?

OOP was created to solve the problems of procedural programming as software grew larger. Procedural code had data and functions separate, making it hard to maintain and reuse. OOP bundles data and behaviour together in objects, making code modular, reusable, and easier to manage at scale.

2 Class and Object

What Is a Class?

A **class** is a **blueprint** or **template**. It defines what attributes (data) and methods (behaviour) an object will have. A class itself does nothing — it's just the design.

What Is an Object?

An **object** is an **instance** of a class — the actual thing created from the blueprint. You can create many objects from one class, each with their own data.

Concept	Analogy	Technical meaning
Class	Cookie cutter / Blueprint / Car design	Template that defines attributes and methods
Object	The actual cookie / The built house / Your car	An instance created from the class in memory

Class and Object Example

```
# CLASS — the blueprint (defined once)
class Student:
    def __init__(self, name, roll, gpa):
        self.name = name       # attribute
        self.roll = roll       # attribute
        self.gpa  = gpa        # attribute

    def introduce(self):       # method
        print(f"Hi, I am {self.name}, Roll: {self.roll}, GPA: {self.gpa}")

    def is_honours(self):      # method
        return self.gpa >= 3.5


# OBJECTS — created from the class (can create many)
ali  = Student("Ali Ahmed",  "BSDSF24A025", 3.7)   # object 1
sara = Student("Sara Khan",  "BSDSF24A026", 3.9)   # object 2
omar = Student("Omar Malik", "BSDSF24A027", 2.8)   # object 3

ali.introduce()           # Hi, I am Ali Ahmed, Roll: BSDSF24A025, GPA: 3.7
print(sara.is_honours())  # True
print(omar.is_honours())  # False

# Each object has its OWN copy of the data
ali.gpa = 3.8   # only changes ali — sara and omar unaffected
```

Memory Visualisation

Class (Blueprint — defined once in code)

```
Student - name (attribute) - roll (attribute) - gpa (attribute) - introduce() (method) - is_honours() (method)
```

ali (Object 1)

```
name = "Ali Ahmed"
roll = "BSDSF24A025"
gpa = 3.7
```

sara (Object 2)

```
name = "Sara Khan"
roll = "BSDSF24A026"
gpa = 3.9
```

Common Mistake: Confusing the class with the object. The class is the blueprint — it exists once in code. Objects are created from it — there can be thousands. Modifying one object's data does NOT affect other objects.

Q What is the difference between a class and an object?

A class is a blueprint — it defines what attributes and methods something will have but doesn't hold actual data. An object is an instance of a class — it's the actual thing created in memory with its own specific data. You can create many objects from one class.

Analogy: Class = cookie cutter (the design). Object = the cookie (the actual thing).

Q Can you create multiple objects from the same class?

Yes. A class is just a template. Every time you call ClassName(), Python creates a new, independent object with its own copy of all the attributes. Changing one object's data doesn't affect others.

3 Constructors

What Is a Constructor?

A constructor is a **special method that runs automatically when an object is created**. Its job is to set up (initialise) the object's attributes with starting values.

In Python, the constructor is always named **__init__**.

Why Do Constructors Exist?

Without a constructor, you'd have to manually set every attribute after creating an object — easy to forget, and creates objects in an inconsistent state. The constructor guarantees every object starts properly set up.

Default Constructor

Default Constructor

```
# Default constructor — no parameters (just self)
class Car:
    def __init__(self):          # called automatically when Car() is called
        self.brand = "Unknown"
        self.speed = 0
        self.running = False

my_car = Car()             # __init__ runs automatically here
print(my_car.brand)        # Unknown
print(my_car.running)      # False
```

Parameterised Constructor

Parameterised Constructor

```
# Parameterised constructor — takes values to customise the object
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year  = year
        self.speed = 0          # always starts at 0

    def accelerate(self, amount):
        self.speed += amount

    def __str__(self):
        return f"{self.year} {self.brand} {self.model} at {self.speed} km/h"

# Must provide all three arguments now
civic  = Car("Honda",  "Civic",  2022)
corolla = Car("Toyota", "Corolla", 2023)

civic.accelerate(60)
print(civic)     # 2022 Honda Civic at 60 km/h
print(corolla)   # 2023 Toyota Corolla at 0 km/h
```

Default Parameter Values

Default Parameter Values

```
# Constructor with default values — flexible
class Student:
    def __init__(self, name, gpa=0.0, semester=1):
```

```
        self.name     = name
        self.gpa      = gpa
        self.semester = semester

# All three ways work:
s1 = Student("Ali", 3.7, 5)    # all provided
s2 = Student("Sara", 3.9)      # semester defaults to 1
s3 = Student("Omar")           # gpa=0.0, semester=1
```

Q What is a constructor? Why do we use it?

A constructor is a special method (__init__ in Python) that is called automatically when an object is created. It initialises the object's attributes with starting values. We use it to ensure every object is properly set up from the moment it's created, preventing inconsistent or incomplete objects.

Q What is the difference between __init__ and __new__ in Python?

__new__ creates the object (allocates memory). __init__ initialises it (sets attributes). __new__ runs first, then __init__. In practice, you almost always only override __init__. __new__ is only needed for advanced cases like the Singleton pattern.

Q What is self in Python?

self refers to the current instance of the class. When you call ali.introduce(), Python automatically passes ali as the first argument — that's self. It lets the method know which object's data to work with. In C++, the equivalent is the this pointer.

4 The Four Pillars of OOP

These four concepts are the foundation of OOP. Every serious OOP interview will ask about them. Know each one deeply: definition, analogy, code, and why it exists.

PILLAR 1

ENCAPSULATION

Definition: Bundling data and methods together in a class, and restricting direct access to internal data.

Analogy: A medicine capsule — the chemicals are sealed inside. You take the pill without touching the contents.

Why: Protects data from accidental or invalid changes. Only the class controls its own data.

Encapsulation Example

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner    = owner
        self.__balance = balance   # __ makes it private

    def deposit(self, amount):
        if amount > 0:             # validation — protects the data
            self.__balance += amount

    def get_balance(self):         # controlled read access
        return self.__balance

    # Python property — clean getter/setter
    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, value):
        if value >= 0:
            self.__balance = value
        else:
            raise ValueError("Balance cannot be negative")

acc = BankAccount("Ali", 5000)
acc.deposit(2000)
print(acc.balance)     # 7000 — accessed through property
# acc.__balance        # Error! Cannot access directly
acc.balance = -100     # raises ValueError — validation works
```

Q What is encapsulation? Why is it important?

Encapsulation means bundling data and the methods that operate on it together in a class, and hiding the internal state from outside access.

It's important because: (1) Data integrity — you can validate data before storing it, (2) Flexibility — you can change internal logic without breaking code that uses the class, (3) Reduces complexity — users only see what they need to.

Common Confusion: Encapsulation is about hiding DATA. Many students confuse it with abstraction (which hides COMPLEXITY). Remember: Encapsulation = data protection.

PILLAR 2

ABSTRACTION

Definition: Hiding complex implementation details and exposing only what is necessary to the user.

Analogy: A TV remote — you press Volume Up without knowing about infrared signals or circuit boards.

Why: Manages complexity. The user doesn't need to know HOW something works, just WHAT it does.

Abstraction Example

```
from abc import ABC, abstractmethod

# Abstract class — defines WHAT must be done, not HOW
class Shape(ABC):
    @abstractmethod
    def area(self):      # must be implemented by every Shape
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def describe(self):  # concrete method — shared by all
        return f"Area={self.area():.2f}, Perimeter={self.perimeter():.2f}"

# Concrete classes — provide the HOW
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        import math
        return math.pi * self.radius ** 2
    def perimeter(self):
        import math
        return 2 * math.pi * self.radius

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):      return self.w * self.h
    def perimeter(self): return 2 * (self.w + self.h)

# Shape()   # TypeError! Cannot instantiate abstract class
c = Circle(5)
r = Rectangle(4, 6)
print(c.describe())   # Area=78.54, Perimeter=31.42
print(r.describe())   # Area=24.00, Perimeter=20.00
```

Q What is abstraction?

Abstraction means hiding complex implementation details and showing only the essential features to the user. In Python, we achieve it using Abstract Base Classes (ABC) with @abstractmethod. The abstract class defines WHAT must be done; subclasses define HOW.

Example: When you call list.sort(), you don't need to know the sorting algorithm — that complexity is hidden. That's abstraction.

PILLAR 3	INHERITANCE
Definition:	A child class automatically gets (inherits) all attributes and methods of a parent class.
Analogy:	A child inherits traits from their parents. They have their own personality but share family traits.
Why:	Code reuse — write once in the parent, all children get it automatically.

Inheritance Example

```
# PARENT CLASS (Base / Superclass)
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age  = age

    def eat(self):
        return f"{self.name} is eating"

    def sleep(self):
        return f"{self.name} is sleeping"

    def speak(self):            # will be overridden
        return "..."


# CHILD CLASSES (Subclasses) — inherit eat() and sleep() automatically
class Dog(Animal):
    def speak(self):            # override parent method
        return f"{self.name} says: Woof!"

    def fetch(self):            # new method — only for Dog
        return f"{self.name} fetches the ball"


class Cat(Animal):
    def speak(self):
        return f"{self.name} says: Meow!"


dog = Dog("Rex", 3)
cat = Cat("Whiskers", 5)

print(dog.eat())    # Rex is eating   (inherited from Animal)
print(dog.speak())  # Rex says: Woof! (overridden in Dog)
print(dog.fetch())  # Rex fetches the ball (Dog-specific)
print(cat.eat())    # Whiskers is eating (inherited)
print(cat.speak())  # Whiskers says: Meow! (overridden)
```

super() — Calling the Parent's Method

super() usage

```
class Animal:
    def __init__(self, name):
        self.name = name
```

```
        print(f"Animal created: {name}")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)   # call Animal.__init__ first
        self.breed = breed       # then add Dog-specific setup
        print(f"Dog breed: {breed}")

rex = Dog("Rex", "German Shepherd")
# Output:
# Animal created: Rex
# Dog breed: German Shepherd
```

Q What is inheritance? What is the benefit?

Inheritance allows a child class to acquire all the attributes and methods of a parent class. The child can also add new methods or override existing ones.

Benefits: (1) Code reuse — write once in parent, all children get it, (2) Logical hierarchy — models real-world relationships, (3) Extensibility — add new types without changing existing code.

PILLAR 4

POLYMORPHISM

Definition: 'Many forms' — the same method name behaves differently depending on the object calling it.

Analogy: The word 'make' means different things: make food, make money, make a call. Same word, different action.

Why: Flexibility — write one piece of code that works with many different object types.

Polymorphism Example

```
# RUNTIME POLYMORPHISM — same method name, different behaviour
class Shape:
    def area(self): pass

class Circle(Shape):
    def __init__(self, r):    self.r = r
    def area(self):           return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self):           return self.w * self.h

class Triangle(Shape):
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self):           return 0.5 * self.b * self.h

# Polymorphism in action — same code works for ALL shapes
shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]

for shape in shapes:
    print(f"Area: {shape.area()}")   # .area() means different things each time

# Output:
# Area: 78.5
# Area: 24
# Area: 12.0
```

```
# DUCK TYPING — Python-specific polymorphism (no inheritance needed)
class PDFDocument:
    def render(self): return "Rendering PDF..."

class WordDocument:
    def render(self): return "Rendering Word Doc..."

class ImageFile:
    def render(self): return "Rendering Image..."

files = [PDFDocument(), WordDocument(), ImageFile()]
for f in files:
    print(f.render())   # works! All have render() — duck typing
```

Q What is polymorphism?

Polymorphism means 'many forms' — the same method name can behave differently depending on which object calls it. It allows one interface to be used for different data types.

In Python, this is achieved through method overriding (child overrides parent's method) and duck typing (if an object has the required method, it works regardless of its type).

Q What is duck typing?

Duck typing is a Python concept: "If it walks like a duck and quacks like a duck, treat it as a duck." An object is compatible with a function if it has the required method — regardless of what class it is. No inheritance needed. Python checks for method existence at runtime, not compile time.

5 Inheritance Types

Python supports all five types of inheritance. These come up in interviews — know the names and be able to draw a quick diagram for each.

Single Inheritance

Animal → Dog

One child inherits from one parent. The most common and simplest type.

```
class Animal: pass
class Dog(Animal): pass    # Dog inherits from Animal only
```

Multiple Inheritance

Father + Mother → Child

One child inherits from multiple parents. Powerful but can cause the diamond problem.

```
class Father:
    def work(self): return "Working"
class Mother:
    def cook(self): return "Cooking"
class Child(Father, Mother):  # inherits from BOTH
    pass

c = Child()
print(c.work())   # Working  (from Father)
print(c.cook())   # Cooking  (from Mother)
```

Multilevel Inheritance

Animal → Mammal → Dog

A chain of inheritance: grandparent → parent → child. Each level adds to the previous.

```
class Animal:
    def breathe(self): return "Breathing"
class Mammal(Animal):
    def feed_young(self): return "Feeding young"
class Dog(Mammal):
    def bark(self): return "Woof!"

rex = Dog()
print(rex.breathe())     # from Animal
print(rex.feed_young())  # from Mammal
print(rex.bark())        # from Dog
```

Hierarchical Inheritance

Vehicle → Car, Bike, Truck

One parent has multiple children. Each child inherits from the same parent independently.

```
class Vehicle:
    def move(self): return "Moving"

class Car(Vehicle):
    def honk(self): return "Beep!"

class Bike(Vehicle):
    def balance(self): return "Balancing"

class Truck(Vehicle):
    def carry(self): return "Carrying load"
```

Hybrid Inheritance

A → B, C → D (diamond)

A combination of two or more inheritance types. Python handles the diamond problem via MRO.

```
class A:
    def hello(self): return "Hello from A"

class B(A): pass   # single
class C(A): pass   # single

class D(B, C):     # multiple — creates diamond shape
    pass

print(D.__mro__)   # shows resolution order
# D → B → C → A → object
```

Interview tip: Single, multiple, and multilevel are the most commonly tested. Always be able to draw the hierarchy arrow diagram for each type. Mention MRO when discussing multiple inheritance in Python.

6 Polymorphism — Deep Dive

Type	Also called	Decided at	How in Python
Method Overriding	Runtime polymorphism	Runtime (when program runs)	Child redefines parent method
Method Overloading	Compile-time polymorphism	Compile time	NOT native in Python — use *args or defaults
Operator Overloading	Ad-hoc polymorphism	Compile time	Define __add__, __eq__, etc.
Duck Typing	Structural polymorphism	Runtime	Any object with the right method works

Method Overriding (Runtime Polymorphism)

The child class provides its own version of a method already defined in the parent. The correct version is chosen at runtime based on the actual object type.

Method Overriding

```
class Payment:
    def process(self, amount):
        pass

class CreditCard(Payment):
    def process(self, amount):
        return f"Credit card charged: PKR {amount}"

class EasyPaisa(Payment):
    def process(self, amount):
        return f"EasyPaisa transferred: PKR {amount}"

class JazzCash(Payment):
    def process(self, amount):
        return f"JazzCash sent: PKR {amount}"

# Runtime polymorphism — type decided when code runs
payments = [CreditCard(), EasyPaisa(), JazzCash()]
for p in payments:
    print(p.process(1000))   # different behaviour each time
```

Method Overloading in Python

Python does NOT support traditional method overloading (same name, different parameters). The last definition wins. Use default parameters or *args instead:

Method Overloading Simulation in Python

```
# Python way to simulate overloading
class Calculator:
    def add(self, *args):      # accepts any number of arguments
        return sum(args)

    def power(self, base, exp=2):   # default parameter
        return base ** exp

calc = Calculator()
print(calc.add(1, 2))         # 3
```

```
print(calc.add(1, 2, 3, 4))   # 10
print(calc.power(3))          # 9  (3 squared)
print(calc.power(2, 10))      # 1024
```

Operator Overloading

Operator Overloading

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):    # overloads + operator
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):     # overloads == operator
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2          # calls __add__
print(v3)             # Vector(4, 6)
print(v1 == v1)       # True — calls __eq__
```

Q What is the difference between method overloading and method overriding?

Overriding: A child class redefines a method from the parent class. Same name, same parameters, different implementation. Decided at RUNTIME based on object type. This IS supported in Python.

Overloading: Multiple methods with the same name but different parameters in the SAME class. Decided at COMPILE TIME. Python does NOT support this natively — use *args or default values.

7 Abstraction vs Encapsulation — The Clear Distinction

This is one of the MOST COMMONLY confused pairs in OOP interviews. Many students give the same answer for both. This section makes it crystal clear.

ENCAPSULATION	ABSTRACTION
Hiding DATA	**Hiding COMPLEXITY**
Bundles data and methods. Restricts direct access to internal state.	Hides how something works. Shows only what the user needs to interact with.
HOW: private/protected attributes (__name), properties, getters/setters	**HOW:** Abstract classes, interfaces (@abstractmethod)
Analogy: A capsule sealing medicine inside	**Analogy:** A car's steering wheel hiding the engine mechanics
Goal: Data integrity and security	**Goal:** Simplicity and managing complexity

Memory Tricks

■ **Encapsulation = Enclosing data**	Think of a locked vault — data locked inside, controlled access
■ **Abstraction = Abstract interface**	Think of an ATM — you see buttons and screen, not the server code behind it

Both Working Together

```
from abc import ABC, abstractmethod

# ABSTRACTION — hides HOW area is calculated
class Shape(ABC):
    @abstractmethod
    def area(self): pass   # just declares WHAT must exist

class Circle(Shape):
    def __init__(self, r):
        self.__radius = r     # ENCAPSULATION — radius hidden

    @property
    def radius(self):         # ENCAPSULATION — controlled access
        return self.__radius

    @radius.setter
    def radius(self, value):  # ENCAPSULATION — validation
        if value > 0: self.__radius = value
        else: raise ValueError("Radius must be positive")

    def area(self):           # ABSTRACTION — hides the formula
        import math
        return math.pi * self.__radius ** 2

# Both work together:
```

```
c = Circle(5)
print(c.area())    # abstraction: you call area(), formula hidden
c.radius = 7       # encapsulation: setter validates the value
# c.__radius = -1  # Error! encapsulation prevents this
```

Q What is the difference between abstraction and encapsulation?

Encapsulation is about hiding DATA — protecting internal state by restricting direct access using private attributes and providing controlled access through methods.

Abstraction is about hiding COMPLEXITY — defining what an object does without revealing how it does it. Achieved through abstract classes and interfaces.

Short answer for interviews: Encapsulation = hide data. Abstraction = hide implementation. A car hides its fuel injection logic (abstraction). Its engine internals are not accessible to the driver (encapsulation).

8 Aggregation vs Composition (HAS-A Relationship)

Both are 'HAS-A' relationships — one object contains another. The difference is about **ownership and lifetime**.

AGGREGATION Weak HAS-A	COMPOSITION Strong HAS-A
Child can exist independently of parent	Child CANNOT exist without parent
Lifetime: Independent — if parent deleted, child survives	**Lifetime:** Dependent — child destroyed when parent is destroyed
Example: Department HAS Students (students exist even if dept closes)	**Example:** House HAS Rooms (rooms cannot exist without the house)
Code: Receive object as parameter	**Code:** Create object INSIDE __init__

Aggregation vs Composition in Code

```
# AGGREGATION — Department HAS Students (weak ownership)
class Student:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, name):
        self.name = name
        self.students = []       # empty — students added later

    def add_student(self, student):  # student passed in from outside
        self.students.append(student)

# Students created independently — they exist before the department
ali  = Student("Ali")
sara = Student("Sara")
cs = Department("CS")
cs.add_student(ali)
cs.add_student(sara)
# If cs is deleted, ali and sara still exist


# COMPOSITION — House HAS Rooms (strong ownership)
class Room:
    def __init__(self, room_type):
        self.room_type = room_type

class House:
    def __init__(self, address):
        self.address = address
        # Rooms created INSIDE House — they belong to this house only
        self.rooms = [Room("Bedroom"), Room("Kitchen"), Room("Bathroom")]

my_house = House("123 Gulberg, Lahore")
# If my_house is deleted, its rooms are gone too
```

Q What is the difference between aggregation and composition?

Both are HAS-A relationships. The difference is ownership and lifetime.

Aggregation (weak HAS-A): the child can exist independently. University HAS students — if the university closes, students still exist as people.

Composition (strong HAS-A): the child cannot exist without the parent. House HAS rooms — a 'room' without a house doesn't make sense as a physical entity.

Code rule: if you CREATE the child object INSIDE __init__, it's composition. If you RECEIVE it as a parameter, it's usually aggregation.

9 Abstract Class vs Interface

Feature	Abstract Class	Interface (in Python: pure ABC)
Methods	Can have abstract AND concrete methods	All methods are abstract (no implementation)
Attributes	Can have instance variables	Typically no instance variables
Inheritance	Single inheritance preferred	Multiple interfaces can be implemented
Purpose	Shared base with some common behaviour	Pure contract — defines WHAT must exist
Python syntax	class X(ABC): with some @abstractmethod	class X(ABC): ALL @abstractmethod
Instantiation	Cannot be instantiated directly	Cannot be instantiated directly

Abstract Class vs Interface

```
from abc import ABC, abstractmethod

# ABSTRACT CLASS — has some implementation
class Vehicle(ABC):
    def __init__(self, brand):     # concrete: has __init__
        self.brand = brand

    def start(self):               # concrete method
        return f"{self.brand} starting..."

    @abstractmethod
    def fuel_type(self):           # abstract: must be overridden
        pass

    @abstractmethod
    def max_speed(self):
        pass


# INTERFACE (pure ABC) — all abstract, no implementation
class Flyable(ABC):
    @abstractmethod
    def fly(self): pass

    @abstractmethod
    def land(self): pass

class Swimmable(ABC):
    @abstractmethod
    def swim(self): pass


# Class can implement multiple interfaces + extend one abstract class
class FlyingCar(Vehicle, Flyable):
    def fuel_type(self):   return "Electric"
    def max_speed(self):   return 300
    def fly(self):         return "Flying at 300km/h"
    def land(self):        return "Landing safely"

fc = FlyingCar("Tesla")
print(fc.start())       # Tesla starting... (from abstract class)
print(fc.fly())         # Flying at 300km/h
```

Q What is an abstract class? When would you use it?

An abstract class is a class that cannot be instantiated directly. It can have both abstract methods (no implementation — must be overridden by children) and concrete methods (with implementation).

Use an abstract class when: you have a group of related classes that share some common behaviour but each must also provide specific implementations. Example: Shape — all shapes have area() but calculate it differently, and they share describe() behaviour.

Q Does Python have interfaces? How do you create one?

Python doesn't have a built-in interface keyword. However, you can create one using ABC where ALL methods are @abstractmethod — no concrete implementation at all.

Multiple interfaces can be 'implemented' through Python's multiple inheritance. A class implementing two interfaces: class Duck(Flyable, Swimmable): ...

10 Access Modifiers

Access modifiers control which parts of a program can access an attribute or method. Python handles them differently from C++ or Java.

Modifier	Python syntax	C++/Java equivalent	Who can access?	Strictly enforced?
Public	self.name	public name	Everyone — any code	N/A — default
Protected	self._name	protected name	Class + subclasses (by convention)	NO — just a hint
Private	self.__name	private name	Only the class itself	Partially — name mangling

Access Modifiers in Python

```
class Employee:
    def __init__(self, name, salary, secret_code):
        self.name         = name          # PUBLIC — anyone can access
        self._department  = "CS"          # PROTECTED — convention: subclasses only
        self.__salary     = salary        # PRIVATE — name mangled to _Employee__salary
        self.__secret     = secret_code  # PRIVATE

    def get_salary(self):                 # controlled public access to private data
        return self.__salary

    def _internal_code(self):             # protected method — convention only
        return "Internal logic here"


class Manager(Employee):
    def show_dept(self):
        return self._department   # OK — protected accessible in subclass

    def show_salary(self):
        return self.get_salary()  # OK — use public method
        # return self.__salary   # Error! Private not accessible in child


emp = Employee("Ali", 80000, "XK9-SECRET")

print(emp.name)            # Ali       — public, fine
print(emp._department)     # CS        — works but bad practice
# print(emp.__salary)      # AttributeError! name mangled
print(emp.get_salary())    # 80000     — correct way

# Python name mangling — private becomes _ClassName__attribute
print(emp._Employee__salary)  # 80000 — technically accessible but NEVER do this
```

Key Point for Interviews: Python does not TRULY enforce private access like C++ or Java. Double underscore (__) uses name mangling — it renames __salary to _Employee__salary — making accidental access harder but not impossible. Python trusts developers to follow conventions.

Q What are access modifiers? How does Python handle them?

Access modifiers control visibility of attributes and methods. Python has three levels:

Public (no underscore): accessible anywhere. Protected (_name): convention only — accessible in class and subclasses but not enforced by Python. Private (__name): uses name mangling — renamed to _ClassName__name, making direct external access harder.

Unlike C++ or Java, Python does not truly enforce private access — it relies on developer convention and name mangling as a deterrent, not a hard restriction.

11 SOLID Principles

SOLID is a set of five design principles for writing clean, maintainable OOP code. They come up in mid-level and senior interviews. As a beginner, focus on understanding what each letter stands for and being able to explain it simply.

S — Single Responsibility Principle

A class should have ONE reason to change. One class = one job.

```
# BAD — UserService does too many things
class UserService:
    def create_user(self): ...
    def send_email(self): ...
    def save_to_db(self): ...
    def generate_report(self): ...

# GOOD — each class has one responsibility
class UserService:
    def create_user(self): ...

class EmailService:
    def send_email(self): ...

class UserRepository:
    def save_to_db(self): ...
```

O — Open/Closed Principle

Open for EXTENSION, closed for MODIFICATION. Add features by adding new code, not changing existing code.

```
# BAD — add new shape = must edit this class
class AreaCalc:
    def area(self, shape):
        if shape == "circle": ...
        elif shape == "rect": ...
        # Adding triangle means modifying AreaCalc!

# GOOD — add new shape = add new class, nothing changes
class Circle(Shape):
    def area(self): ...

class Triangle(Shape):  # new class — no existing code changed
    def area(self): ...
```

L — Liskov Substitution Principle

A subclass must be replaceable for its parent without breaking the program.

```
# VIOLATION — Square breaks Rectangle contract
class Rectangle:
    def set_width(self, w):  self.w = w
    def set_height(self, h): self.h = h
    def area(self): return self.w * self.h

class Square(Rectangle):    # bad: overrides break contract
    def set_width(self, w):  self.w = self.h = w

# GOOD — separate classes or common abstract base
class Shape(ABC):
```

```
    @abstractmethod
    def area(self): pass
```

I — Interface Segregation Principle

Don't force a class to implement methods it doesn't need. Many small interfaces > one large one.

```
# BAD — forces Robot to implement manage_budget
class Worker(ABC):
    @abstractmethod
    def work(self): pass
    @abstractmethod
    def manage_budget(self): pass  # robots cant do this!

# GOOD — separate focused interfaces
class Workable(ABC):
    @abstractmethod
    def work(self): pass

class Manageable(ABC):
    @abstractmethod
    def manage_budget(self): pass

class Human(Workable, Manageable): ...
class Robot(Workable): ...
```

D — Dependency Inversion Principle

Depend on abstractions, not concrete classes. Inject dependencies from outside.

```
# BAD — tightly coupled to MySQLDatabase
class OrderService:
    def __init__(self):
        self.db = MySQLDatabase()  # hard-coded dependency

# GOOD — depends on abstraction, injected from outside
class Database(ABC):
    @abstractmethod
    def save(self, data): pass

class OrderService:
    def __init__(self, db: Database):  # injected!
        self.db = db

service = OrderService(MySQLDatabase())  # or any Database
```

12 50 Interview Questions with Full Answers

These are the most commonly asked OOP questions at internship and entry-level interviews. Study the answer structure — start with a definition, give an analogy, then give an example.

1 What is OOP?

A programming paradigm that organises code around objects — units that bundle data (attributes) and behaviour (methods) together. Makes code modular, reusable, and easier to maintain.

2 What are the four pillars of OOP?

Encapsulation (hide data), Inheritance (reuse and extend), Polymorphism (many forms), Abstraction (hide complexity). Memory trick: EIPA or 'Every Intelligent Programmer Abstracts'.

3 What is a class?

A blueprint or template that defines what attributes and methods an object will have. Exists once in code. Does not hold actual data itself — just the structure.

4 What is an object?

An instance of a class — the actual thing created in memory from the blueprint. Can create many objects from one class, each with its own data.

5 What is the difference between a class and an object?

Class = blueprint (cookie cutter). Object = instance (the cookie). One class, many objects. Each object has independent data.

6 What is a constructor?

A special method (__init__ in Python) called automatically when an object is created. Sets up initial attribute values. Ensures every object starts in a valid state.

7 What is self in Python?

Refers to the current instance. Passed automatically as first argument to instance methods. Equivalent to this pointer in C++.

8 What is the difference between __init__ and __new__?

__new__ creates the object (memory allocation). __init__ initialises it (sets attributes). __new__ runs first. In practice, only override __new__ for advanced patterns like Singleton.

9 What is encapsulation?

Bundling data and methods together in a class and restricting direct access to internal data. Data is hidden; interaction happens through controlled methods.

10 Why is encapsulation important?

Data integrity (validate before storing), flexibility (change internals without breaking external code), reduced complexity (users see only what they need).

11 What is a getter and setter?

Getter: method to read a private attribute. Setter: method to write it with validation. In Python, use @property decorator for clean getter/setter syntax.

12 What is name mangling in Python?

When you prefix an attribute with __ (double underscore), Python renames it to _ClassName__attr. Makes accidental external access harder. Not truly private — just deterrent.

13 What is inheritance?

Child class automatically gets attributes and methods of parent class. Models is-a relationships. Enables code reuse.

14 What is super()?

Returns a proxy to the parent class. Used to call parent's __init__ or methods from a child class. Ensures parent is properly set up before child adds its own setup.

15 What is method overriding?

Child class provides its own implementation of a method defined in the parent. Same name and parameters, different body. Parent version replaced for that object type.

16 What types of inheritance exist?

Single (one parent), multiple (many parents), multilevel (chain: A→B→C), hierarchical (one parent, many children), hybrid (combination).

17 What is the diamond problem?

In multiple inheritance: if B and C both inherit from A, and D inherits from B and C, which A method does D use? Python solves with MRO (C3 linearisation). Check with __mro__.

18 What is MRO?

Method Resolution Order — the order Python searches for a method in a class hierarchy. Uses C3 linearisation algorithm. View with ClassName.__mro__.

19 When should you use inheritance vs composition?

Inheritance for is-a relationships (Dog IS-A Animal). Composition for has-a relationships (Car HAS-A Engine). Prefer composition — it's more flexible and avoids tight coupling.

20 What is polymorphism?

Many forms — same method name behaves differently depending on which object calls it. One interface, multiple implementations. Achieved through overriding and duck typing.

21 What is runtime vs compile-time polymorphism?

Runtime: method overriding — correct version chosen when program runs based on object type. Compile-time: method overloading — correct version chosen before running based on parameters. Python only has runtime polymorphism natively.

22 Does Python support method overloading?

Not natively. The last definition replaces earlier ones with the same name. Simulate with *args, **kwargs, or default parameter values.

23 What is duck typing?

If an object has the required method, Python treats it as compatible — regardless of class. 'If it walks like a duck and quacks like a duck, it is a duck.' No inheritance required.

24 What is operator overloading?

Defining __add__, __eq__, __lt__ etc. to make + == < work with custom objects. Example: Vector + Vector can be defined by implementing __add__.

25 What is abstraction?

Hiding complex implementation details and showing only the essential interface. WHAT the object does, not HOW it does it. Achieved with ABC and @abstractmethod.

26 What is an abstract class?

A class with at least one @abstractmethod that cannot be instantiated directly. Forces subclasses to provide implementations. Can also have concrete methods.

27 Can you instantiate an abstract class?

No. Attempting to do so raises TypeError. You must create a subclass that implements all abstract methods, then instantiate the subclass.

28 What is @abstractmethod?

A decorator from the abc module that marks a method as abstract. Any class inheriting from an ABC with @abstractmethod must implement that method, or it also becomes abstract and cannot be instantiated.

29 What is the difference between abstraction and encapsulation?

Encapsulation hides DATA (protects internal state). Abstraction hides COMPLEXITY (hides HOW). Encapsulation = vault locking data. Abstraction = ATM hiding server code.

30 What is the difference between association, aggregation, and composition?

Association: objects know about each other (uses-a). Aggregation: weak has-a — child exists independently. Composition: strong has-a — child cannot exist without parent.

31 What is the difference between aggregation and composition?

Aggregation: University HAS students — students exist without university. Composition: House HAS rooms — rooms don't exist without the house. Key: if you create child INSIDE __init__ → composition. Receive it as param → aggregation.

32 What is IS-A vs HAS-A relationship?

IS-A = inheritance. Dog IS-A Animal. HAS-A = composition/aggregation. Car HAS-A Engine. IS-A creates tight coupling — use HAS-A when possible.

33 What is an interface?

A pure contract — defines WHAT methods a class must have, no implementation. Python: ABC where all methods are @abstractmethod. Classes 'implement' by inheriting and defining all methods.

34 What is the difference between abstract class and interface?

Abstract class: can have concrete + abstract methods, instance variables. One inheritance. Interface: all abstract, no implementation, multiple can be implemented. Use abstract class for shared behaviour; interface for pure contract.

35 What is SRP?

Single Responsibility Principle — a class should have only one reason to change. One class = one job. Separating concerns makes code easier to test and maintain.

36 What is OCP?

Open/Closed Principle — open for extension, closed for modification. Add new behaviour through new classes, not by editing existing working code.

37 What is LSP?

Liskov Substitution Principle — a subclass must be replaceable for its parent without breaking the program. If you have to override a method in a way that violates parent's contract, you're breaking LSP.

38 What is ISP?

Interface Segregation Principle — don't force classes to implement methods they don't need. Split large interfaces into smaller, focused ones.

39 What is DIP?

Dependency Inversion Principle — depend on abstractions, not concrete classes. Inject dependencies from outside instead of creating them inside. Makes code testable.

40 What are dunder methods?

Methods with double underscores: __init__, __str__, __add__, __len__ etc. They let your class integrate with Python built-ins. __str__ controls print(). __add__ controls +. __len__ controls len().

41 What is @classmethod vs @staticmethod?

@classmethod receives cls (the class) as first arg. Can access/modify class state. Used for factory methods. @staticmethod receives nothing automatically — just a regular function in the class namespace. No access to class or instance.

42 What is @property in Python?

A decorator that lets you define getter/setter/deleter methods while keeping attribute-style access. Code calls obj.name (not obj.get_name()) but validation runs internally.

43 What is multiple inheritance?

A class can inherit from multiple parents: class Child(Parent1, Parent2). Python handles conflicts via MRO. Use with care — can increase complexity.

44 What is a mixin?

A small class designed to add specific behaviour to other classes via multiple inheritance. Not a full standalone class — just adds methods. Example: LogMixin adds logging to any class.

45 What is tight coupling vs loose coupling?

Tight: class depends directly on a specific concrete implementation. Hard to change, test, or replace. Loose: class depends on abstractions. Easy to swap implementations. Achieved via DIP and interfaces.

46 What is dependency injection?

Instead of a class creating its own dependencies internally, they are passed from outside. Makes code testable (can inject mocks) and flexible (can change implementations).

47 What is the difference between __str__ and __repr__?

__str__: human-readable output, called by print() and str(). __repr__: developer-readable, called by repr(). Should uniquely identify the object. If only one is defined, Python falls back to __repr__.

48 What is the Singleton pattern?

Ensures only ONE instance of a class exists. Override __new__ to check if instance exists. Return existing instance if it does. Used for database connections, config managers.

49 What does 'favour composition over inheritance' mean?

Instead of using inheritance to add behaviour, use composition — contain objects of other classes. Composition is more flexible: you can change behaviour at runtime by swapping components. Inheritance creates tighter coupling that's harder to change.

50 Can a child class access parent's private attributes?

Not directly. Private attributes use name mangling (__attr becomes _ClassName__attr). A child class trying to access self.__salary gets AttributeError because Python looks for _ChildClass__salary which doesn't exist. Use protected (_attr) for child access.

51 What is method resolution order (MRO) and how does it work?

MRO defines the order Python searches for methods in a class hierarchy. Uses C3 linearisation. For class D(B, C): Python checks D → B → C → A → object. Always check left-to-right, then up the hierarchy.

13 10-Minute Pre-Interview Cheat Sheet

Read this page the morning of your interview. It covers everything you need to remember.

The 4 Pillars — One Line Each

ENCAPSULATION	Bundle data + methods. Hide internal state. Controlled access via getters/setters.
ABSTRACTION	Hide HOW. Show only WHAT. Use ABC + @abstractmethod. Cannot instantiate abstract class.
INHERITANCE	Child gets parent's attributes + methods. is-a relationship. Use super() for parent init.
POLYMORPHISM	Same method, different behaviour. Override in child (runtime). Duck typing in Python.

Most Confused Pairs — At a Glance

Encapsulation vs Abstraction	Enc = hide DATA (private attrs). Abs = hide COMPLEXITY (abstract methods). Vault vs ATM.
Inheritance vs Composition	Inheritance = is-a (Dog IS Animal). Composition = has-a (Car HAS Engine). Prefer composition.
Overriding vs Overloading	Override = child changes parent method (runtime). Overload = same name diff params (Python: use *args).
Abstract Class vs Interface	Abstract: some concrete methods + abstract. Interface: ALL abstract. Python: both use ABC.
Aggregation vs Composition	Aggregation = weak has-a (child lives independently). Composition = strong (child dies with parent).
== vs is	== checks value equality. is checks identity (same object in memory). a=[1]; b=[1] → a==b True, a is b False.

SOLID — One Line Each

S	**Single Responsibility**	One class = one job = one reason to change
O	**Open/Closed**	Add features by extending, not by editing existing code
L	**Liskov Substitution**	Subclass must work wherever parent is used without breaking things
I	**Interface Segregation**	Many small focused interfaces > one giant interface
D	**Dependency Inversion**	Depend on abstractions not concrete classes; inject dependencies

Access Modifiers Quick Reference

Python	Type	Who accesses	Enforced?
self.name	Public	Anyone	Yes (default)
self._name	Protected	Class + subclasses	No — convention only
self.__name	Private	Class only	Partial — name mangling

How to Answer Any OOP Question in an Interview

Formula: 1. One-sentence definition → 2. Real-world analogy → 3. Code or example → 4. Why it matters / benefit → 5. Python-specific note if relevant

Example for Encapsulation: 'Encapsulation means bundling data and methods in a class and restricting direct access to internal data — like a capsule sealing medicine inside. In Python, we use __ for private attributes. The benefit is data integrity — you can validate before storing, so no invalid values like GPA = -5 can be set.'

14 Interview Master Revision Table

Use this table as your final scan before walking into any interview.

Concept	Definition	Why It Exists	Interview Keywords
Class	Blueprint/template defining attributes and methods	Structure code; define what objects will look like	*blueprint, template, definition*
Object	Instance of a class — actual data in memory	Represent real entities with their own state	*instance, instantiate, new*
Constructor	__init__ — runs automatically when object created	Guarantee objects start in valid, set-up state	*__init__, initialise, self*
Encapsulation	Hide data; expose controlled interface	Data integrity and flexibility	*private, protected, getter, setter, __attr*
Abstraction	Hide HOW; show only WHAT	Manage complexity; define contracts	*abstract, ABC, @abstractmethod, interface*
Inheritance	Child gets parent attributes and methods	Code reuse; model is-a relationships	*extends, super(), override, is-a*
Polymorphism	Same method name, different behaviour per type	Flexibility; write once for many types	*override, duck typing, many forms*
Overriding	Child redefines parent method	Specialise behaviour per subclass	*runtime, virtual, same signature*
Overloading	Same name, different parameters (not native in Python)	One method, multiple uses	*compile-time, *args, defaults*
Association	Objects know each other; uses-a	Model relationships without ownership	*uses-a, loose coupling*
Aggregation	Weak has-a; child exists independently	Model collections without ownership	*has-a, weak, independent lifetime*
Composition	Strong has-a; child dies with parent	Model tight ownership	*has-a, owns-a, strong, nested*
Abstract Class	Cannot instantiate; has abstract + concrete methods	Force subclasses to implement; share common code	*ABC, abstractmethod, cannot instantiate*
Interface	Pure contract; all abstract, no implementation	Define what a class MUST do	*pure abstract, contract, implement*
Public	No underscore — accessible anywhere	Default access; expose safe attributes	*open, accessible*
Protected	Single _ — convention: subclasses only	Communicate 'internal use' intent	*_name, convention, not enforced*
Private	Double __ — name mangled to _Class__attr	Prevent accidental external access	*__name, name mangling, _Class__attr*
SRP	One class = one responsibility	Maintainability; easy to test	*single job, separation of concerns*
OCP	Extend without modifying existing code	Stability; open to growth, closed to bugs	*extend, new class, don't edit*

LSP	Subclass replaceable for parent	Correctness in polymorphic code	*substitutable, contract, no violation*
ISP	Many small interfaces over one large	Avoid forcing unnecessary implementations	*segregate, focused, small interfaces*
DIP	Depend on abstractions, inject dependencies	Testability and flexibility	*inject, abstract, inversion*
Duck Typing	If it has the method, it works — no inheritance required	Pythonic flexibility; loose coupling	*EAFP, duck, method exists, dynamic*
MRO	Method Resolution Order — C3 linearisation	Resolve multiple inheritance conflicts	*MRO, __mro__, C3, diamond problem*
Singleton	Only one instance ever created	Shared resource management	*__new__, one instance, global state*

You now have everything you need for any OOP interview.

The formula for every answer: Definition → Analogy → Example → Benefit → Python note.

Practice saying answers out loud. An answer you can write is not the same as one you can explain confidently under pressure. Talk through the concepts daily.

Good luck.